Feedback-Driven Personalization

A proposal for training a recommendation model from stored dashboard feedback

Bonus deliverable — proposal only, not implemented

Project: Crypto Advisor
Author: Roy Meoded
Document date: August 2026

1. What we already store

Every thumbs-up/thumbs-down click in the dashboard is already persisted in `content_feedback`, one row per user per piece of content:

```
content_feedback(user_id, section_type, content_key, vote, created_at)
user_preferences(user_id, interested_assets, investor_type, content_types)
```

This is exactly the raw material a recommendation model needs: **who** (investor type, selected assets), reacted **how** (vote), to **what** (section_type + content_key). No new instrumentation would be required to start collecting a training set — it is already flowing in through normal use of the app.

2. The problem today

Personalization currently uses one fixed, hand-written rule: a user's saved `content_types` preference nudges a matching section earlier in the dashboard (see `dashboard-ordering.ts`). This rule is the same for every user who picked the same preference — it does not learn from whether that user (or similar users) actually engaged positively with what they were shown. A trained model could replace or blend with this rule once enough feedback accumulates.

3. Two different problems, two different techniques

Not all four dashboard sections behave the same way, and the proposal should not force one algorithm onto all of them. The `content_key` format already tells us which technique fits which section:

Section	`content_key` pattern	Shared across users?	Best-fit technique
Market News	`news:<provider>:<article-id>`	Yes — same article shown to many users	Collaborative filtering (item-based)
Fun Crypto Meme	`meme:<meme-id>`	Yes — finite curated catalog	Collaborative filtering (item-based)
Coin Prices	`prices:<assets>:<date>`	No — unique per user/day	Content-attribute classifier
AI Insight	`insight:<daily-insight-id>`	No — unique per user/day	Content-attribute classifier

News articles and memes are drawn from a shared, finite pool — the same `content_key` genuinely repeats across different users' feedback rows, which is exactly what collaborative filtering needs ("users who upvoted this article/meme also upvoted..."). Coin-price snapshots and AI insights, by contrast, are generated fresh per user per day and never repeat verbatim — a classic

recommendation model has nothing to collaborate on there. Those two are better served by a classifier trained on the *attributes* of the content (which assets, which investor-type tone, which content category) rather than on the specific (non-reusable) item id.

3.1 Collaborative filtering — Market News & Crypto Meme

A standard implicit-feedback matrix-factorization approach (e.g. a lightweight ALS or a lookalike model) over a user × content_key matrix of votes. Even with a modest user base, this lets the app start ranking "already-fetched" news articles by predicted relevance instead of showing them in raw fetch order, and rotate memes toward ones similar users engaged with positively.

3.2 Content-attribute classifier — Coin Prices & AI Insight

A small supervised classifier (logistic regression or gradient-boosted trees are both a reasonable starting point given the expected data volume) predicting `P(thumbs-up)` from features derived from `user_preferences` (investor_type, interested_assets, content_types) and content attributes (e.g. which asset the price row is about, the insight's tone/length). This would not recommend a specific historical insight again — it would inform *how future insights are generated* (e.g. adjusting prompt tone per investor_type based on which past tones scored well) and which assets to prioritize surfacing.

4. Proposed data flow

```
content_feedback + user_preferences  (already collected)
        |
        v
  offline batch export  (scheduled job, not on the request path)
        |
        v
  feature engineering  (per section-type, as above)
        |
        v
  train / validation split, held-out votes for evaluation
        |
        v
  model artifact (versioned, stored outside the request path)
        |
        v
  served at read time: blended with (not replacing) the existing
  preference-based ordering rule, so a cold model never makes the
  dashboard worse than today's baseline
```

5. Cold start

A brand-new user has no feedback history yet. The existing preference-based heuristic (already implemented and tested) is the natural cold-start fallback — the model would only take over

incrementally, per user, once that user (and similar users, for the collaborative-filtering half) has produced enough signal to be predicted with confidence above the heuristic baseline.

6. What would be needed before this is worth building

- Meaningfully more votes than the app has collected so far — a handful of rows per user is not enough to train or evaluate any of the above without overfitting.
- A decision on evaluation metric (e.g. precision on held-out thumbs-up votes) and a minimum bar the model must clear before it's allowed to override the existing heuristic.
- A scheduled retraining job and a place to store versioned model artifacts — infrastructure this project deliberately does not include yet (see the project's scope-control rules against premature infrastructure).

7. Scope note

This is a written proposal only, as requested by the assignment's bonus item — no training code, model, or infrastructure is included in this submission. The goal is to demonstrate that the current data model already supports this future direction without needing to be redesigned.

> This document is a design proposal for a possible future feature. Nothing described here is implemented in the current version of Crypto Advisor.